

Unit 1 Exercise: Complexity Analysis

This is our first out of class exercise, which means you are expected to work on these exercises outside of class time. Once you have completed the exercises and is confident in your answers – then you will enter your answer on D2L under the Tests section in the section labelled Unit 1 out of class exercise.

Note: You have only one attempt at this “quiz”, if you’re unsure of your answers, please check with your instructor during class time.

Unit 1 Overview

Unit 1 is about understanding what algorithms are and how algorithms behave — not just for small inputs, but as input sizes grow. Our focus is on efficiency, scalability, and making better algorithmic decisions. By the end of this lab, you should be able to analyze runtime, compare algorithms, and connect theory to real-world performance.

Exercise Overview

In this exercise, we focus on four key ideas. First, understanding how algorithms scale as input size increases. Second, comparing algorithms that solve the same problem but with different time complexities. Third, applying Big-O notation to simplify and analyze performance. Finally, connecting theory to practice by calculating maximum input sizes under time constraints. The goal is not just solving equations, but building intuition about algorithm efficiency.

Learning Goals

You will be able to recognize linear, quadratic, and exponential growth patterns. You will estimate maximum input sizes based on time constraints, simplify runtime expressions into Big-O notation, and analyze code structures such as loops.

Why Complexity Matters

At small input sizes, many algorithms appear similar. But as input sizes grow, differences become dramatic. This is scalability. Linear algorithms can handle large inputs well, while exponential algorithms quickly become impractical. Hardware improvements

cannot compensate for poor algorithm choices, so long-term growth matters more than initial speed.

Comparing Growth Rates

We will compare linear, quadratic, and exponential growth. Linear growth scales proportionally. Quadratic growth increases much faster. Exponential growth doubles rapidly and quickly become infeasible. This understanding helps you choose the right algorithm depending on constraints.

Section A: Solving for Maximum Input Size

You will translate runtime expressions into maximum input sizes. Convert time into seconds, set runtime equal to time limit, solve for n , and round down. Linear equations are straightforward, quadratic require square roots, and exponential require logarithms. This connects algebra with real computing limits.

Examples – First Table

Note: all running times are given in seconds.

Algorithm A in 1 sec:

$$0.2n = 1 \text{ second}$$

$$n = 1 / 0.2$$

$$n = 5$$

Algorithm A in 1 min:

$$0.2n = 60 \text{ seconds}$$

$$n = 60 / 0.2$$

$$n = 300$$

Algorithm A in 1 hour:

$$0.2n = 3600 \text{ seconds}$$

$$n = 3600 / 0.2$$

$$n = 18000$$

Algorithm B in 1 sec:

$$0.02n^2 = 1 \text{ second}$$

$$n^2 = 1 / 0.02$$

$$n = \sqrt{1 / 0.02}$$

$$n = 7.0710678...$$

*n must be an integer, and we must round down to measure units of data processed!

$$\text{floor}(n) = 7$$

Algorithm C in 1 sec:

Remember to use log base 2 calculator!

$$0.0002 * 2^n = 1 \text{ second}$$

$$2^n = 1 / 0.0002$$

$$n = \log \text{ base } 2 (1 / 0.0002)$$

$$n = \log \text{ base } 2 (5000)$$

$$n = 12.288$$

$$\text{floor}(n) = 12$$

Follow the same formula for the rest of the table.

Example – Second Table

Algorithm A in 1 sec:

$$0.02n = 1 \text{ second}$$

$$n = 1 / 0.02$$

$$n = 50$$

Follow the same formula for the rest of the table.

Interpreting Results

After solving the section, the key is interpretation. Growth rate dominates constant factors. Faster hardware helps simple algorithms predictably, but offers little benefit to exponential ones. Time constraints limit feasible input sizes.

Section B: Big-O Simplification

Big-O notation simplifies complexity by focusing on the dominant term. Remove constants, ignore lower-order terms, and keep only the fastest growing part. This allows meaningful comparison between algorithms.

Dominant Terms

To simplify expressions, identify which term grows fastest as n increases. Logarithmic grows slower than linear, and exponential grows faster than polynomial. This becomes intuitive with practice and allows faster analysis.

Examples:

15: no n , constant growth rate $\rightarrow O(1)$

$0.6n^2$: remove constants $\rightarrow O(n^2)$

$6 \log n + 0.001$: dominant term = $6 \log n$, remove constant = $\log n \rightarrow O(\log n)$

Σ - Summations

1) $1 + 2 + 3 + 4 + \dots + n \rightarrow n(n+1)/2$

2) $1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2 \rightarrow n(n+1)(2n+1)/6$

Section C: Ordering Growth Rates

You will order complexity classes from slowest to fastest in terms of runtime – how long it takes for the algorithm to finish running and produce the correct answer.

Simplify each term, then convert each to its corresponding big O notation. Place them in order of their performance.

Section D: Recognizing Patterns

Learn to recognize complexity directly from code. Loops that halve values suggest logarithmic complexity. Nested loops suggest polynomial growth. This helps estimate complexity quickly in real-world code.

Loop Complexity

A single loop from 0 to n is $O(n)$. Nested loops multiply complexity, leading to $O(n^2)$ and higher. Break code into parts, analyze each, and combine results.

Logarithmic and Mixed Complexity

Loops that divide values each iteration produce $O(\log n)$. Combining with linear loops results in $O(n \log n)$. These patterns appear in efficient algorithms such as merge sort.

Bit-shift Operators

Operators $\gg$ and $\ll$ are referred to as bit shift operators. These operators exist in many programming languages for fast and precise manipulations of data. They operate on the binary representation of the value.

- 1) $\ll$: left-shift, push every bit over to the left by one position, fill rightmost empty position with 0
- 2) $\gg$: right-shift, push every bit over to the right by one position, fill leftmost empty position with 0

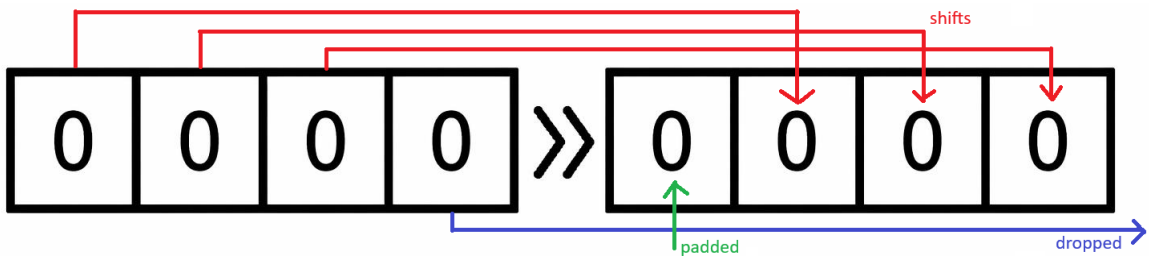
What does Bit-shift Operators do?

Let's start with a few numbers in the most common numbering systems:

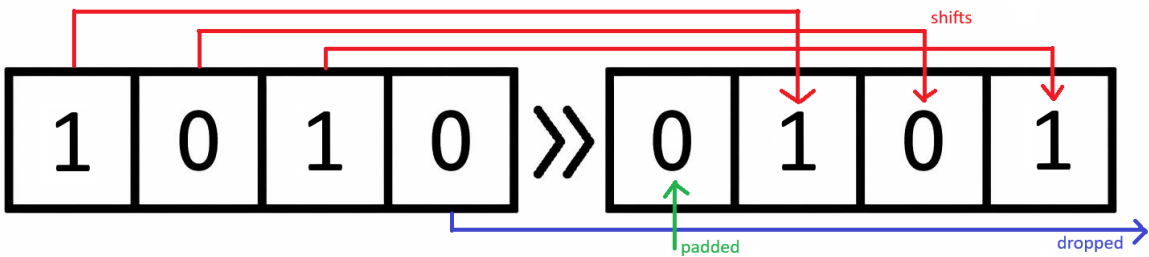
Decimal	Hexadecimal	Binary (4-bit)
0	0	0000
1	1	0001
2	2	0010

3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

E.g. binary 0000:



E.g. binary 1010:



Decimal	Original Binary	Right-shifted Binary	Shifted Decimal	Right-shift Effect
0	0000	0000	0	0/2 = 0
1	0001	0000	0	1/2 = 0
2	0010	0001	1	2/2 = 1
3	0011	0001	1	3/2 = 1
4	0100	0010	2	4/2 = 2
5	0101	0010	2	5/2 = 2
6	0110	0011	3	6/2 = 3
7	0111	0011	3	7/2 = 3

8	1000	0100	4	$8/2 = 4$
9	1001	0100	4	$9/2 = 4$
10	1010	0101	5	$10/2 = 5$
11	1011	0101	5	$11/2 = 5$
12	1100	0110	6	$12/2 = 6$
13	1101	0110	6	$13/2 = 6$
14	1110	0111	7	$14/2 = 7$
15	1111	0111	7	$15/2 = 7$

Right-shifting is the binary version of dividing by 2!

This is much easier and faster to do by the processor (CPU)! One of the simplest optimizations we can do as a programmer!

What did you learn?

To summarize: focus on growth rates, simplify correctly, connect math to real performance, and build intuition through practice. These exercises will help you think like a rockstar software developer when evaluating algorithms and data structures for any software applications!

You can now start critically compare and analyze any tool, package, framework, <insert any new tech of the day> etc.! Keep these fundamentals in mind and you will be able to dig deeper and figure out the “best” thing for whatever you need to do!